# Late Breaking Result: AQFP-aware Binary Neural Network Architecture Search

Zhengang Li[1], Xuan Shen[1], Geng Yuan[2], Masoud Zabihi[1], Tomoharu Yamauchi[3], Yanzhi Wang[1], Olivia Chen[3]

[1]Northeastern University [2]University of Georgia [3]Tokyo City University

[1]{li.zhen, shen.xu, yanz.wang}@northeastern.edu, [3]olivia.chen@ieee.org

## ABSTRACT

Adiabatic Quantum-Flux-Parametron (AQFP) is a superconducting logic with extremely high energy efficiency. Recent research has made initial strides toward developing AQFP accelerator. However several critical challenges from both the hardware and software side remain, preventing the design from being a comprehensive solution. This paper proposes an AQFP-aware binary neural network architecture search framework that leverages software-hardware co-optimization to eventually search the AQFP-adapted neural network and the corresponding hardware configuration, providing a feasible AQFP-based solution for binary neural network (BNN) acceleration. Experimental results show that our framework consistently outperforms the representative AQFP-based framework.

**ACM Reference Format:**

Zhengang Li[1], Xuan Shen[1], Geng Yuan[2], Masoud Zabihi[1], Tomoharu Yamauchi[3], Yanzhi Wang[1], Olivia Chen[3] . 2024. Late Breaking Result: AQFP-aware Binary Neural Network Architecture Search. In *61st ACM/IEEE Design Automation Conference (DAC '24), June 23–27, 2024, San Francisco, CA, USA.* ACM, New York, NY, USA, 2 pages. https://doi.org/10.1145/3649329.3663503

## 1 INTRODUCTION

Superconducting electronics are considered one of the most promising future computing devices due to their high energy efficiency. In 1985, the quantum-flux-parametron (QFP)-based superconducting logic device was first proposed [6], which made Josephson-Junction (JJ) logic device working on the principle of parametron with DC flux. On top of the QFP logic design, an adiabatic version of QFP was proposed [10], as known as the AQFP logic devices, which re-parameterized the device to make QFP gates work at an adiabatic mode. AQFP logic devices realize an extremely low energy dissipation that is roughly 5-6 orders lower than its CMOS counterpart. Recent work SupeRBNN [5] proposes an AQFP-aware binary neural network (BNN) training framework accounting for AQFP characteristics. However, since it only considers the AQFP randomized behavior, the overall performance of BNN implementation remains constrained due to the unsuitable model architecture.

A well-designed neural network for other types of technologies may not work for AQFP-based accelerators because of the existence of the randomized behavior [5] and the limited AQFP devices' scalability. Therefore, to fully unleash the potential of AQFP-based BNN accelerators, it is desirable to have a dedicated and comprehensive neural network design under the characteristics of AQFP devices.

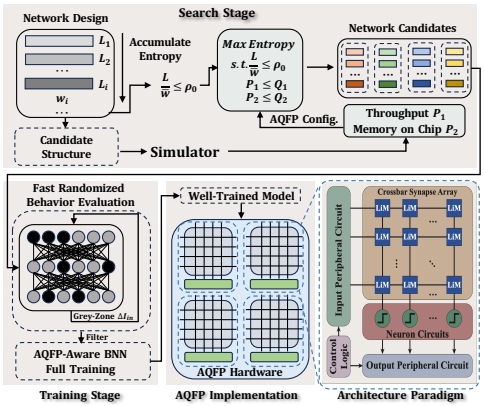


**Figure 1: Overview of the AQFP-aware BNN search.**

To this end, **we propose the first AQFP-aware BNN search framework, which can automatically and efficiently design the best-suited BNN architecture within a range of available AQFP hardware configurations and train the corresponding high-performance BNN model accordingly.** Specifically, our framework consists of a *Search Stage* and a *Training Stage*. We propose a novel AQFP-aware zero-shot neural architecture search (NAS) algorithm that employs the entropy of a candidate network architecture to evaluate its expressiveness and ensure the capacity and accuracy of the neural network in a zero-shot manner accelerating the search process. Our algorithm also imports the AQFP configurations, such as target throughout and the chip memory, as the constraints to search the desirable neural network architecture. We achieve 92.6% accuracy with 1.3 MB model size and 86.6% accuracy with only 33 KB model size on Cifar-10 dataset. Compared with the MobileNet-V2, EfficinetNet-B0, and current AQFP BNN framework [5], our searched model is more adapted to AQFP computation and achieves better accuracy with less model size.

## 2 METHODOLOGY

**Architecture Paradigm:** To efficiently handle the data movement issue in traditional Von Neumann architectures, we follow the logic-in-memory (LiM) array-based crossbar proposed in paper [12] as shown at the lower right corner of Figure 1. The binarized weights are stored in 1-bit AQFP LiM cells and multiplied by the in-cell XNOR macro. Unlike conventional popcount-based accumulation in BNNs, the crossbar employs an analog summation approach. This approach directly adds up all the outputs since positive and negative current pulses represent the logic '1' and '0' in AQFP. Depending on the direction of the accumulated current, the AQFP buffers in the neural circuit serve as a sign function and an ADC to convert the accumulated current into the binary value "0" or "1".



*DAC '24, June 23–27, 2024, San Francisco, CA, USA*



ACM ISBN 979-8-4007-0601-1/24/06

https://doi.org/10.1145/3649329.3663503

**Overview of the AQFP-aware BNN Search:** Here we present the overview of our proposed framework as shown in Figure 1. The entire paradigm consists of two stages: the AQFP-aware BNN search stage and the AQFP-aware BNN training/implementation stage. In the first stage, we generate the model architecture based on information entropy transforming the entire search process into pure mathematical programming. Inspired by the computation cost of CNN models, we can define the width of one convolution with $c_i k_i^2/g_i$, where $c_i$, $k_i$, and $g_i$ denote the number of input channels, kernel size, and group number separately. Then, we deliver the expressiveness (i.e., entropy) of CNN in Equation 1 with the effectiveness (i.e., depth-to-width ratio) in Equation 2. The $r_{L+1}$ and $c_{L+1}$ denote the resolution of output feature maps and the number of output channels at $L$-th convolutional layer, and log computation is applied for the constraint of the same magnitude in computation.

$$H_L = \log(r_{L+1}^2 c_{L+1}) \sum_{i=1}^{L} \log(c_i k_i^2/g_i), \tag{1}$$

$$\rho = L \cdot (\prod_{i=1}^{L} w_i)^{-1/L}, \quad w_i = c_i k_i^2/g_i, \tag{2}$$

As the networks will be implemented on the AQFP devices, we also design the budgets to incorporate the AQFP energy efficiency, throughput, and scalability into our NAS algorithm, which helps the NAS algorithm generate more AQFP-friendly networks. Considering that the AQFP crossbar size significantly affects model accuracy and hardware performance [5], we generate architecture candidates based on different crossbar sizes, guided by an AQFP simulator that estimates the throughput and layer/model-wise memory consumption. Considering the efficiency of model inference on AQFP devices, the throughput of the model is restricted as the budget $Q_1$. Also, the scalability of the networks is restricted by the pre-store memory of the AQFP devices as the budget $Q_2$. The whole mathematical programming problem is denoted as Equation 3, which can be easily solved by off-the-shelf solvers for constrained non-linear programming [9]:

$$\begin{aligned} \max_{w_i, L_i} \quad & \sum_{i=1}^{M} \alpha_i H_i \quad \text{s.t.} L \cdot (\prod_{i=1}^{L} w_i)^{-1/L} \le \rho_0, \\ \textit{and} \quad & Throughput[f(\cdot)] \le Q_1, Memory[f(\cdot)] \le Q_2, \end{aligned} \tag{3}$$

After the AQFP-aware network architectures are generated, we filter them with AQFP-aware BNN training [5]. The architecture candidates and the corresponding crossbar size are sent to the training module and processed with early stopping for fast randomized behavior evaluation according to the accuracy reducing the cost of time and resources. A series of different widths of "gray-zone" $\Delta I_{in}$ is also combined in the training process to evaluate accuracy performance. After a couple of epochs of AQFP-aware BNN training, it can distinguish the relative accuracy of different candidates. Ultimately, we perform full AQFP-aware BNN training for the best candidate, consisting of the best BNN architecture, crossbar size, and the "gray-zone" setting.

## 3 EXPERIMENTAL EVALUATION

The implementation of AQFP hardware uses a semi-automated design strategy targeting the AIST 4-layer 10 kA/cm$^2$ niobium process (HSTP) [7]. We conduct circuit-level verification using a thermal noise-considering Josephson simulator Jsim [2] modification. Based on testing data, we build AQFP memory bank simulator, with each bank's memory model adhering to CACTI [1], comprising 4 Mats, each containing four crossbars. Table 1 shows the energy efficiency comparison across different technologies with different optimization schemes. The baselines incorporate CMOS-based SyncBNN [3], ReRAM-based IMB [4], superconducting-based ERSFQ [3], SupeRBNN [5]. Due to the cooling energy dissipation required by the superconducting devices, we also compare the energy efficiency with cooling. Table 2 shows the model performance on the Cifar-10 dataset comparing our AQFP-aware searched network with binarized representative efficient models, including MobileNet-V2 [8], and EfficientNet-B0 [11]. We also train a larger model to compare with the results from SupeRBNN [5]. We achieve an extremely small model with 33 KB and 86.6% accuracy. The conventional efficient models such as EfficientNet-B0 and MobileNet-V2 show poor accuracy after binarization. Compared with SupeRBNN [5], our result achieves better accuracy with shorter latency under the same energy efficiency.

**Table 1: Comparison of the energy efficiency across different technologies with different optimization schemes.**

| Tech. | Scheme | Energy Efficiency (TOPS/W) W/O Cooling | W/ Cooling |
|---|---|---|---|
| IMB [4] | Binary | 82.6 | - |
| SyncBNN [3] | Binary | 36.6 | 12.03 |
| ERSFQ [3] | Binary | $1.5 \times 10^4$ | 50.0 |
| SupeRBNN [5] | Binary | $3.8 \times 10^5$ | $9.5 \times 10^2$ |
| **Ours** | **Binary** | $\mathbf{3.8 \times 10^5}$ | $\mathbf{9.5 \times 10^2}$ |

**Table 2: Searched model performance analysis and comparison on Cifar-10 dataset.**

| Model | Accuracy (%) | Model Size (KB) | Latency ($\mu$s) |
|---|---|---|---|
| EfficientNet-B0 [11] | 72.1 | 450 | 7.3 |
| MobileNet-V2 [8] | 83.8 | 287 | 20.5 |
| **Ours** | 86.6 | 33 | 8.8 |
| SupeRBNN (VGG) [5] | 90.6 | $1.9 \times 10^3$ | 256.4 |
| **Ours** | 92.6 | $1.3 \times 10^3$ | 230.3 |

## 4 CONCLUSION

In this paper, we propose the framework for the binary neural network architecture design on AQFP hardware devices. This search framework leverages software-hardware co-optimization and generates the AQFP-adapted networks.

## ACKNOWLEDGMENTS

This work was supported by JST FOREST Program (Grant Number JPMJFR226W, Japan) and JSPS KAKENHI Grant Number JP23H03365.

## REFERENCES

[1] [n. d.]. https://www.hpl.hp.com/research/cacti/.
[2] Emerson S. Fang. 1989. A Josephson integrated circuit simulator (JSIM) for superconductive electronics application.
[3] Rongliang Fu. 2022. JBNN: A Hardware Design for Binarized Neural Networks Using Single-Flux-Quantum Circuits. *IEEE Trans. Comput.* (2022).
[4] Hyungjun Kim et al. 2019. In-memory batch-normalization for resistive memory based binary neural network hardware.
[5] Zhengang Li et al. 2023. SupeRBNN: Randomized Binary Neural Network Using Adiabatic Superconductor Josephson Devices. *arXiv:2309.12212* (2023).
[6] K. Loe and E. Goto. 1985. Analysis of flux input and output Josephson pair device. *IEEE Transactions on Magnetics* (1985).
[7] S. Nagasawa. 2005. Reliability evaluation of Nb 10 kA/cm2 fabrication process for large-scale SFQ circuits. *Physica C* (2005).
[8] Mark Sandler et al. 2018. Mobilenetv2: Inverted residuals and linear bottlenecks. In *CVPR*.
[9] Xuan Shen et al. 2023. DeepMAD: Mathematical Architecture Design for Deep Convolutional Neural Network. In *CVPR*.
[10] Naoki Takeuchi et al. 2013. An adiabatic quantum flux parametron as an ultra-low-power logic device. (2013).
[11] Mingxing Tan and Quoc Le. 2019. Efficientnet: Rethinking model scaling for convolutional neural networks. In *ICML*.
[12] Tomoharu Y. 2023. Design and Implementation of Energy-Efficient Binary Neural Networks Using Adiabatic Quantum-Flux-Parametron Logic. *TAS* (2023).